

Pixel- and Image-Based Pathfinding Algorithms: Design, Pseudocode, and Incremental Enhancements

Shaimaa Said Soltan

June 1, 2026

Contents

1	Introduction	2
2	Pixel-Based Maze Solver	2
2.1	Concept	2
2.2	Pseudocode	2
2.3	Enhancements and Advancement	4
3	Longest-White-Path Solver	5
3.1	Concept	5
3.2	Pseudocode (Heuristic DFS-Based Longest Path)	5
3.3	Enhancements and Advancement	5
4	16-Direction BFS Solver	6
4.1	Concept	6
4.2	Pseudocode	6
4.3	Enhancements and Advancement	6
5	Grid-Based Neighbor Solver	6
5.1	Concept	6
5.2	Pseudocode	6
5.3	Enhancements and Advancement	7
6	Image-Based Traversal Solver	7
6.1	Concept	7
6.2	Pseudocode (Dijkstra on Image)	7
6.3	Enhancements and Advancement	7
7	Left/Right/Up/Down Directional Solver	7
7.1	Concept	7
7.2	Pseudocode (BFS with Direction State)	8
7.3	Enhancements and Advancement	8
8	Overall Progression and Enhancements	8

1 Introduction

This document describes a family of pathfinding and traversal algorithms designed for road networks and maze-like images, all implemented in a pixel- or grid-based setting. Each section presents:

- The core idea of the algorithm.
- Detailed pseudocode.
- Enhancements and how it advances beyond previous variants.

We assume a 2D discrete domain (image or rasterized road network) with width W and height H , indexed by integer coordinates (x, y) , $0 \leq x < W$, $0 \leq y < H$.

2 Pixel-Based Maze Solver

2.1 Concept

The pixel-based maze solver treats the rasterized road network (or maze) as a binary image: *road* pixels (white) and *non-road* pixels (black). A standard breadth-first search (BFS) is performed on the 4-neighborhood (up, down, left, right) of each pixel to find a shortest path between a start pixel s and a target pixel t .

2.2 Pseudocode

We assume a function `isRoad(x,y)` that returns `true` if pixel (x, y) is a valid road pixel.

2.3 Enhancements and Advancement

- **Enhancement:** Using a rasterized representation of roads allows direct BFS on pixels, avoiding explicit graph construction.
- **Advancement:** This algorithm is the baseline for all subsequent methods. Later algorithms extend it with more directions, different neighbor models, or additional constraints (e.g., longest path, directional bias).

Algorithm 1 Pixel-Based Maze Solver (4-Connected BFS)

Require: Start pixel $s = (s_x, s_y)$, target pixel $t = (t_x, t_y)$

Require: Image dimensions W, H

Ensure: Path P as a sequence of pixels from s to t , or empty if none

```
1: Initialize visited[0..W*H-1] to false
2: Initialize prev[0..W*H-1] to -1
3: Initialize queue  $Q$  as empty
4:  $idx_s \leftarrow s_y \cdot W + s_x$ 
5: visited[idx_s]  $\leftarrow$  true
6: Enqueue  $s$  into  $Q$ 
7: Define neighbor offsets:
8:    $\Delta = \{(1, 0), (-1, 0), (0, 1), (0, -1)\}$ 
9:  $found \leftarrow false$ 
10: while  $Q$  is not empty do
11:    $(x, y) \leftarrow$  Dequeue from  $Q$ 
12:   if  $(x, y) = (t_x, t_y)$  then
13:      $found \leftarrow true$ 
14:     break
15:   end if
16:   for all  $(dx, dy) \in \Delta$  do
17:      $nx \leftarrow x + dx, ny \leftarrow y + dy$ 
18:     if  $0 \leq nx < W$  and  $0 \leq ny < H$  then
19:        $idx \leftarrow ny \cdot W + nx$ 
20:       if visited[idx] = false and isRoad(nx, ny) then
21:         visited[idx]  $\leftarrow$  true
22:         prev[idx]  $\leftarrow y \cdot W + x$ 
23:         Enqueue  $(nx, ny)$  into  $Q$ 
24:       end if
25:     end if
26:   end for
27: end while
28: if not  $found$  then
29:   return empty path
30: end if
31: // Reconstruct path
32:  $P \leftarrow$  empty list
33:  $idx \leftarrow t_y \cdot W + t_x$ 
34: while  $idx \neq -1$  do
35:    $x \leftarrow idx \bmod W$ 
36:    $y \leftarrow \lfloor idx / W \rfloor$ 
37:   Prepend  $(x, y)$  to  $P$ 
38:    $idx \leftarrow prev[idx]$ 
39: end while
40: return  $P$ 
```

3 Longest-White-Path Solver

3.1 Concept

Instead of finding the *shortest* path between two points, the longest-white-path solver aims to find a path that maximizes the number of white (road) pixels visited, subject to connectivity constraints. This is generally a hard problem (longest path in a graph is NP-hard), so we use a heuristic or constrained variant.

A simple variant: perform a BFS/DFS from a start pixel and track the longest simple path discovered under a depth or time limit.

3.2 Pseudocode (Heuristic DFS-Based Longest Path)

Algorithm 2 Heuristic Longest-White-Path Solver

Require: Start pixel $s = (s_x, s_y)$

Require: Image dimensions W, H

Ensure: A long (not necessarily optimal) path P

```
1:  $bestPath \leftarrow$  empty list
2: Define neighbor offsets:
3:    $\Delta = \{(1, 0), (-1, 0), (0, 1), (0, -1)\}$ 
4: Initialize  $visited[0..W*H-1]$  to false
5: function DFS( $x, y, currentPath$ )
6:   Append ( $x, y$ ) to  $currentPath$ 
7:    $idx \leftarrow y \cdot W + x$ 
8:    $visited[idx] \leftarrow$  true
9:   if  $|currentPath| > |bestPath|$  then
10:      $bestPath \leftarrow currentPath$ 
11:   end if
12:   for all  $(dx, dy) \in \Delta$  do
13:      $nx \leftarrow x + dx, ny \leftarrow y + dy$ 
14:     if  $0 \leq nx < W$  and  $0 \leq ny < H$  then
15:        $nidx \leftarrow ny \cdot W + nx$ 
16:       if  $visited[nidx] = \text{false}$  and  $\text{isRoad}(nx, ny)$  then
17:         DFS( $nx, ny, currentPath$ )
18:       end if
19:     end if
20:   end for
21:    $visited[idx] \leftarrow$  false
22:   Remove last element from  $currentPath$ 
23: end function
24: DFS( $s_x, s_y$ , empty list)
25: return  $bestPath$ 
```

3.3 Enhancements and Advancement

- **Enhancement:** Instead of minimizing path length, this solver explores the space of simple paths to maximize coverage of white pixels.

- **Advancement:** This algorithm extends the pixel-based maze solver by changing the objective from shortest path to longest path, illustrating how the same underlying grid representation can support different optimization goals.

4 16-Direction BFS Solver

4.1 Concept

The 4-connected BFS only allows moves in cardinal directions. The 16-direction BFS solver extends the neighbor set to include diagonals and additional intermediate directions, approximating a more continuous movement model and reducing path jaggedness.

We define a set of 16 direction vectors (dx_i, dy_i) , each representing a possible step.

4.2 Pseudocode

Algorithm 3 16-Direction BFS Solver

Require: Start pixel s , target pixel t , image dimensions W, H

Ensure: Shortest path in terms of number of steps (in 16-connected grid)

- 1: Initialize **visited**, **prev**, and queue Q as in Section 2.
 - 2: Define 16-direction neighbor offsets:
 - 3: $\Delta \leftarrow \{(1, 0), (-1, 0), (0, 1), (0, -1),$
 - 4: $(1, 1), (1, -1), (-1, 1), (-1, -1),$
 - 5: $(2, 1), (2, -1), (-2, 1), (-2, -1),$
 - 6: $(1, 2), (1, -2), (-1, 2), (-1, -2)\}$
 - 7: Run BFS exactly as in Algorithm 1, but using Δ above.
 - 8: Reconstruct path using **prev** as before.
-

4.3 Enhancements and Advancement

- **Enhancement:** The neighbor set is expanded from 4 to 16 directions, allowing more natural diagonal and shallow-angle moves.
- **Advancement:** Compared to the pixel-based maze solver, this algorithm produces smoother, less Manhattan-like paths and better approximates Euclidean shortest paths on the raster.

5 Grid-Based Neighbor Solver

5.1 Concept

The grid-based neighbor solver abstracts the image into a coarser grid (e.g., superpixels or tiles). Each cell in the coarse grid represents a block of pixels, and neighbors are defined at the cell level. This reduces the search space and can accelerate pathfinding on large images.

5.2 Pseudocode

Assume we partition the image into cells of size $k \times k$ pixels. Let (i, j) index cells, with $0 \leq i < W_c$, $0 \leq j < H_c$.

Algorithm 4 Grid-Based Neighbor Solver

Require: Coarse grid dimensions W_c, H_c , cell size k

Require: Start cell S_c , target cell T_c

Ensure: Path as a sequence of cells (optionally refined to pixels)

- 1: Define `cellIsRoad(i, j) = true` if the cell contains at least one road pixel.
 - 2: Initialize `visited[0..WcHc-1]` and `prev[0..WcHc-1]` as before.
 - 3: Define neighbor offsets on the coarse grid:
 - 4: $\Delta_c = \{(1, 0), (-1, 0), (0, 1), (0, -1)\}$
 - 5: Run BFS on cells (i, j) using `cellIsRoad` and Δ_c .
 - 6: Reconstruct cell-level path P_c from S_c to T_c .
 - 7: Optionally, for each consecutive pair of cells in P_c , refine to a pixel-level path inside the union of the two cells using a local pixel-based BFS.
 - 8: **return** refined path.
-

5.3 Enhancements and Advancement

- **Enhancement:** Coarse grid representation reduces the state space, improving performance on large maps.
- **Advancement:** This algorithm generalizes the pixel-based solver by introducing a multi-resolution view: global routing on a coarse grid, local refinement on pixels.

6 Image-Based Traversal Solver

6.1 Concept

The image-based traversal solver treats the entire image as a weighted graph, where each pixel has a traversal cost derived from image properties (e.g., brightness, color, road thickness). Instead of a binary `isRoad`, we use a cost function $c(x, y)$ and run a Dijkstra-like algorithm or weighted BFS.

6.2 Pseudocode (Dijkstra on Image)

Algorithm 5 Image-Based Traversal Solver (Dijkstra)

Require: Start pixel s , target pixel t , cost function $c(x, y)$

Ensure: Minimum-cost path from s to t

```
1: Initialize distance array dist[0..W*H-1] to  $+\infty$ 
2: Initialize prev[0..W*H-1] to -1
3: Use a priority queue  $PQ$  keyed by distance
4:  $idx_s \leftarrow s_y \cdot W + s_x$ 
5: dist[idx_s]  $\leftarrow$  0
6: Insert  $(0, s_x, s_y)$  into  $PQ$ 
7: Define neighbor offsets (4 or 8 directions):
8:    $\Delta = \{(1, 0), (-1, 0), (0, 1), (0, -1)\}$ 
9: while  $PQ$  not empty do
10:   Extract  $(d, x, y)$  with minimal  $d$  from  $PQ$ 
11:    $idx \leftarrow y \cdot W + x$ 
12:   if  $d > \text{dist}[idx]$  then
13:     continue
14:   end if
15:   if  $(x, y) = t$  then
16:     break
17:   end if
18:   for all  $(dx, dy) \in \Delta$  do
19:      $nx \leftarrow x + dx, ny \leftarrow y + dy$ 
20:     if  $0 \leq nx < W$  and  $0 \leq ny < H$  then
21:        $nidx \leftarrow ny \cdot W + nx$ 
22:        $w \leftarrow c(nx, ny)$ 
23:       if  $\text{dist}[idx] + w < \text{dist}[nidx]$  then
24:          $\text{dist}[nidx] \leftarrow \text{dist}[idx] + w$ 
25:          $\text{prev}[nidx] \leftarrow idx$ 
26:         Insert  $(\text{dist}[nidx], nx, ny)$  into  $PQ$ 
27:       end if
28:     end if
29:   end for
30: end while
31: Reconstruct path from  $t$  using prev as before.
```

6.3 Enhancements and Advancement

- **Enhancement:** Introduces a cost function over pixels, allowing preference for thicker roads, brighter pixels, or specific classes of terrain.
- **Advancement:** Generalizes unweighted BFS to weighted shortest paths, enabling more realistic routing that accounts for road quality, width, or other image-derived attributes.

7 Left/Right/Up/Down Directional Solver

7.1 Concept

The left/right/up/down directional solver is a constrained BFS that only allows moves in the four cardinal directions, but can incorporate directional penalties or preferences. For example, it can penalize frequent turns or prefer forward motion, which is useful for generating turn-minimizing routes.

7.2 Pseudocode (BFS with Direction State)

We augment the state with the current direction $dir \in \{L, R, U, D\}$.

Algorithm 6 Directional BFS with Turn Penalty

Require: Start pixel s , target pixel t

Ensure: Path that balances distance and turn minimization

```
1: Directions:
2:    $D = \{(1, 0) : R, (-1, 0) : L, (0, 1) : D, (0, -1) : U\}$ 
3: For each pixel index  $idx$  and direction  $d$ , maintain  $\text{dist}[idx][d]$  (cost) and  $\text{prev}[idx][d]$ .
4: Initialize all  $\text{dist}$  to  $+\infty$ ,  $\text{prev}$  to null.
5: Use a priority queue  $PQ$  over states  $(cost, x, y, dir)$ .
6: For all initial directions  $d \in \{L, R, U, D\}$ :
7:    $idx_s \leftarrow s_y \cdot W + s_x$ 
8:    $\text{dist}[idx_s][d] \leftarrow 0$ 
9:   Insert  $(0, s_x, s_y, d)$  into  $PQ$ 
10: Turn penalty  $\lambda > 0$ .
11: while  $PQ$  not empty do
12:   Extract  $(c, x, y, dir)$  with minimal  $c$ 
13:    $idx \leftarrow y \cdot W + x$ 
14:   if  $(x, y) = t$  then
15:     break
16:   end if
17:   for all  $(dx, dy, dir2) \in D$  do
18:      $nx \leftarrow x + dx, ny \leftarrow y + dy$ 
19:     if  $0 \leq nx < W$  and  $0 \leq ny < H$  and  $\text{isRoad}(nx, ny)$  then
20:        $nidx \leftarrow ny \cdot W + nx$ 
21:        $\text{turnCost} \leftarrow 0$  if  $dir2 = dir$  else  $\lambda$ 
22:        $\text{newCost} \leftarrow c + 1 + \text{turnCost}$ 
23:       if  $\text{newCost} < \text{dist}[nidx][dir2]$  then
24:          $\text{dist}[nidx][dir2] \leftarrow \text{newCost}$ 
25:          $\text{prev}[nidx][dir2] \leftarrow (idx, dir)$ 
26:         Insert  $(\text{newCost}, nx, ny, dir2)$  into  $PQ$ 
27:       end if
28:     end if
29:   end for
30: end while
31: Choose the best direction  $d^*$  at  $t$  with minimal  $\text{dist}[idx_t][d]$  and reconstruct the path using  $\text{prev}$ .
```

7.3 Enhancements and Advancement

- **Enhancement:** Incorporates direction as part of the state and penalizes turns, allowing control over route smoothness and turn count.
- **Advancement:** Extends the basic 4-connected BFS by embedding directional preferences, bridging the gap between pure shortest-path routing and navigation-style routing that cares about turn minimization.

8 Overall Progression and Enhancements

The algorithms form a progression:

1. **Pixel-based maze solver:** Baseline unweighted BFS on a binary road image.
2. **Longest-white-path solver:** Changes the objective from shortest to longest path, exploring coverage.
3. **16-direction BFS solver:** Increases directional resolution, producing smoother paths.
4. **Grid-based neighbor solver:** Introduces multi-resolution routing for scalability.
5. **Image-based traversal solver:** Generalizes to weighted costs derived from image properties.
6. **Left/right/up/down directional solver:** Adds direction to the state and penalizes turns, aligning with navigation semantics.

Each step adds either:

- A richer objective (longest path, turn minimization),
- A richer state (direction, cost),
- Or a richer representation (multi-resolution grid, weighted image).